

TECHNISCHE
UNIVERSITÄT
WIEN

Processes, exec, Pipes

191.002 Operating Systems VU
2026W

Luca Di Stefano, Jaroslav Klapalek, Joel Klimont, Felix Resch,
Peter Puschner
2026-11-03

Part I

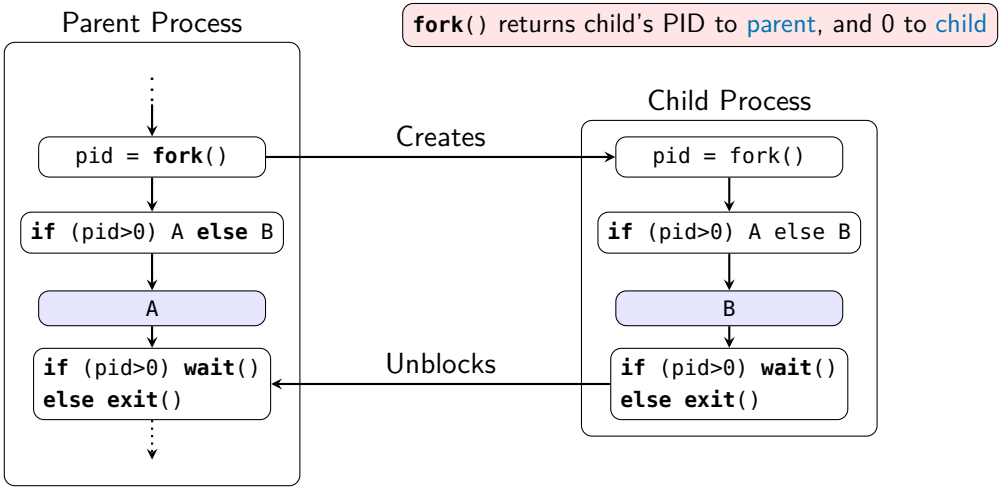
Creating processes

A Refresher on Processes

- A process is how the OS represents a running program
- Every process has a PID (process ID)
- Every process except PID 1 has a parent process
- So far we have worked with unrelated processes
- We can also let a process create related (child) processes as needed

```
$ pstree -p 1
systemd(1)-+-ModemManager(3270)-+-{ModemManager}(3277)
           |                    |-{ModemManager}(3279)
           |                    `-{ModemManager}(3282)
           |-NetworkManager(3167)-+-{NetworkManager}(3227)
           |                      |-{NetworkManager}(3228)
           .
           .
           |-containerd(3465)-+-{containerd}(3470)
           |                  |-{containerd}(3471)
           .
           .
           |-cron(3118)
           |-dbus-daemon(3087)
           |-dockerd(4940)-+-{dockerd}(4947)
           |               |-{dockerd}(4948)
           .
           .
```

Process Creation Overview



Creating a process: `fork(2)`


```c
#include <unistd.h>
pid_t fork(void);
```

Returns:

-1	On error ($\rightarrow$ `errno`)
0	In the child process
>0	In the parent process

Process Creation: Example


```
1  pid_t pid = fork();
2
3  switch (pid) {
4    case -1:
5      fprintf(stderr, "fork failed: %s\n", strerror(errno));
6      exit(EXIT_FAILURE);
7
8    case 0: /* ... Child instructions ... */ break;
9
10   default: /* ... Parent instructions ... */ break ;
11 }
```

Child process properties

Child inherits from parent:

- Opened files (common access!)
- File buffers
- Signal handling
- Current values of variables

But:

- Signal handling can be re-configured independently
- Variables are independent (separate memory space)
- Inter process communication (IPC) needed for sharing data

Pitfalls


```c
#include <stdio.h>

int main(int argc, char **argv) {
    printf("Hello");

    fork();
    return 0;
}
```

Output: "HelloHello". Why?

Answer

Unflushed buffer copied on `fork`!

Pitfalls


```c
#include <stdio.h>

int main(int argc, char **argv) {
    printf("Hello");
    fflush(stdout); // <--- !!!
    fork();
    return 0;
}
```

Output: "Hello"

Remember to flush all opened streams before forking

Terminating a process: `exit(3)`


```
void exit(int status);
```

- `status`: Number in the 0-255 range. Conventionally:
  - `exit(EXIT_SUCCESS)` – process completed successfully
  - `exit(EXIT_FAILURE)` – error occurred
  - (Just FYI) BSD: `sysexits.h`
  - (Just FYI) `http://tldp.org/LDP/abs/html/exitcodes.html`
- Actions performed by `exit()`:
  - Flush and close `stdio` stream buffers
  - Close all open files
  - Delete temporary files (see `man 3 tmpfile`)
  - Call exit handlers (see `man 3 atexit`)

Waiting on Child Processes: `wait(2)`, `waitpid(2)`


```c
#include <sys/wait.h>
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

- `wait()` blocks until a child (*any* child) terminates or an error occurs
- `waitpid()` blocks until the specified child terminates (-1 = *any* child)
- Returns PID of the terminated child, or -1 on error ($\rightarrow$ `errno`)
- `status` will contain additional information
  - `WIFEXITED(status)`: true if child terminated normally
  - `WEXITSTATUS(status)`: exit status of child
  - `WIFSIGNALED(status)`, `WTERMSIG(status)`: information about signals
- `options`: customise behaviour of `waitpid()`

Waiting Examples

```c
#include <sys/wait.h>

int status;
pid_t child_pid, pid;

...
pid = wait(&status);
if (WEXITSTATUS(status) == EXIT_SUCCESS) { /* child 'pid' succeeded */ }

pid = waitpid(-1, &status, 0);                  // Same as pid = wait(&status)

...
child_pid = fork();
if (child_pid) waitpid(child_pid, &status, 0);  // wait for specific child
```

Zombies and Orphans

- Terminated processes stay in OS's process table until parent calls `wait()`
- No more space in process table → no new process can be started!

Zombie Child terminates, but parent didn't call `wait()` yet

- State of the child is set to "zombie"
- Child remains in process table until parent calls `wait()`

Orphan Parent terminates before child

- Child becomes an orphan and is inherited to the `init` process
- When an orphan terminates, the `init` process removes its entry from the process table

Notification of Termination of a Child

Scenario: parent needs to know when a child terminates, but also wants to do other work. Cannot just call `wait()`: would block!

The synchronous way

- `waitpid(-1, &status, WNOHANG)`
- Returns a PID > 0 if a child has terminated, 0 otherwise
- Returns immediately
- Repeating calls $\rightarrow$ polling (can waste CPU time)

The asynchronous way

- Catch `SIGCHLD` signals with a handler (`sigaction`)
- Call `wait` inside the handler (guaranteed not to block)

Part II

The exec Family

exec Family Overview

- Family of calls (from `unistd.h`) to launch an executable
- They load the new program into the same process
  (PID remains the same, but memory is replaced)
- If successful in launching the program, they do not return to the caller!
- The trick: combine with `fork()`

```
int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execle(const char *path, const char *arg, ..., char *const envp[]);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[]);
int fexecve(int fd, char *const argv[], char *const envp[]);
```

exec Family

- `execl•` – variable number of arguments
- `execv•` – arguments via array
- `exec•p` – searching the environment variable `$PATH` for the program specified
- `exec•e` – environment[1] can be changed (others inherit from caller)
- `fexecve` – accepts file descriptor of executable, instead of path

[1] See `man 7 environ`

exec Family: Examples


```c
#include <unistd.h>
```

```c
char *cmd[] = { "ls", "-l", (char *) 0 };
execv("/bin/ls", cmd);
```

```c
execl("/bin/ls", "ls", "-l", NULL);
```

```c
int fd = open("/bin/ls", O_RDONLY);
fexecve(fd, cmd, NULL);
```

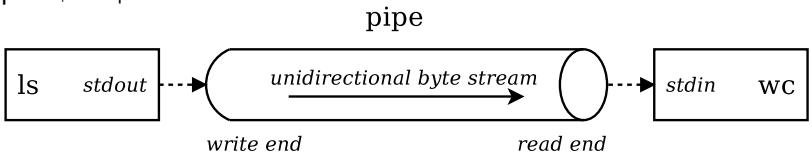
Note on Arguments

- Provide the full argument list, including `argv[0]` (program name)
- Last argument must be a `NULL` pointer!

Part III

Pipes

- Example: `$ ls | wc -l`



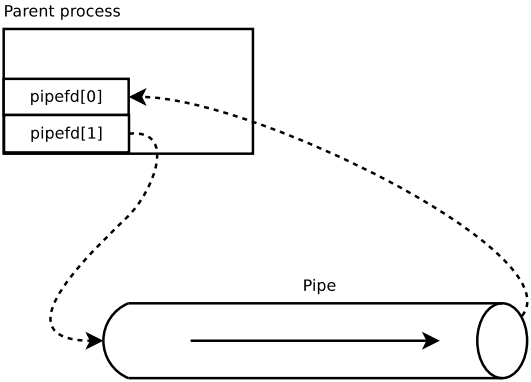
- One-way byte stream
- Access to read and write end of the pipe via file descriptors
- Buffered
- Implicit synchronisation

Creating a pipe: `pipe(2)`


```c
#include <unistd.h>
int pipe(int pipefd[2]);
```

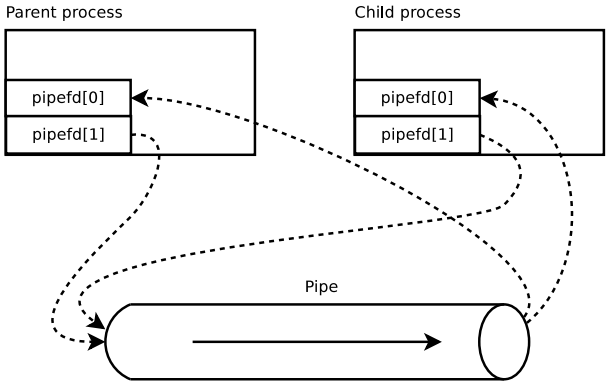
- Places two file descriptors into `pipefd`
  - `pipefd[0]` – read end
  - `pipefd[1]` – write end
- Returns 0 on success, -1 on error ($\rightarrow$ `errno`)

1. Create pipe



Creating and Using Pipes

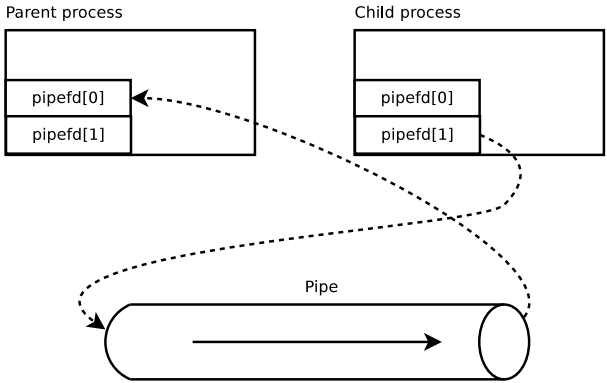
1. Create pipe
2. Call `fork()`



Creating and Using Pipes

1. Create pipe
2. Call `fork()`
3. After `fork()`, each process closes the end it does not need
4. Use `read(2)` and `write(2)`, or wrap into a stream (`fdopen(3)`)

Need 2-way communication?
Create 2 pipes!



Implicit Synchronisation with Pipes

- `read()` blocks when pipe is empty
- `write()` blocks when pipe is full

- `read` indicates end-of-file if all write ends are closed (return value 0)
- `write` creates signal `SIGPIPE` if all read ends are closed
- Writing on closed pipe $\rightarrow$ error `EPIPE`

Closing unused pipe ends is important!

- Lets other process know what is happening
- OS cleans up pipes when all their ends are closed

Named pipes (FYI)

- Unnamed pipes: `pipe` (C), `|` (Shell)
- Named pipes: `mkfifo(1)`, `mknod(2)`
- Similar to files
- However, reading from empty pipe blocks
  (whereas reading from empty file returns EOF)

Redirection of stdin/stdout

Why?

Scenario:

- Parent forks, child invokes another program (exec)
- Parent wants to get child's output, but child prints to `stdout`

Approach:

- Parent: create pipe P, `fork()`
- Child: close read end of P and `stdout`
- Child: make `fd 1` (ie. `stdout`) point to P's write end
- Child: execute program (`exec`)
- Parent: close write end of P and read from its read endian

  (Same for `stdin`, but reversed)

  (Of course, works for any file descriptor)

Duplicating a File Descriptor: `dup(2)`, `dup2(2)`


```c
#include <unistd.h>
int dup(int oldfd);
int dup2(int oldfd, int newfd);
```

`dup(oldfd)` duplicates file descriptor `oldfd`

- New file descriptor uses smallest unused integer
- Not another `open`: points to the same open file description

`dup2`: same but you can specify the new fd

- If `newfd` is open, `dup2` closes it first

Both return new fd on success, -1 on error ($\rightarrow$ `errno`)

Redirection Example


```c
int pipefd[2]; char buffer[1024]; FILE *stream;
pipe(pipefd);

switch(fork()) {
  case -1: /* error handling */ break;
  case 0:       close(pipefd[0]);               // Close read end
                dup2(pipefd[1], STDOUT_FILENO);
                close(pipefd[1]);               // Close old fd
                execlp("ls", "ls", "-l", NULL); break;
  default:      close(pipefd[1]);               // Close write end
                stream = fdopen(pipefd[0], "r");
                while (fgets(buffer, sizeof buffer, stream)) { /* ... */ }
                fclose(stream); wait();
}
```

Pitfalls with Pipes

- Need two pipes for two-way communication
  - Process A writes on pipe P_1 and reads from pipe P_2
  - Process B writes on pipe P_2 and reads from pipe P_1
- What if they both try to read, and both pipes are empty?

  $\rightarrow$ Deadlock!
- Design carefully
- (FYI) `poll(2)`, `fcntl(2)` to do non-blocking checks

Buffered I/O may have surprising effects

- Flush buffers, especially before `fork()`
- (FYI) Buffers may be configured: `setbuf(3)`, `setvbuf(3)`

Tips for the Exercise

- Try to parallelize as much as possible
- Do not exploit copy of memory areas
  (Obvious exceptions: pipes, `argc/argv`, ...)

Do NOT

1. Parent reads entire file into an array, then forks
2. Child processes data from (copy of) array

Instead DO

1. Parent creates pipe and forks, then reads file line-by-line (eg., `fgets()`)
2. Lines are sent to child over a pipe
3. Child processes each line as it arrives

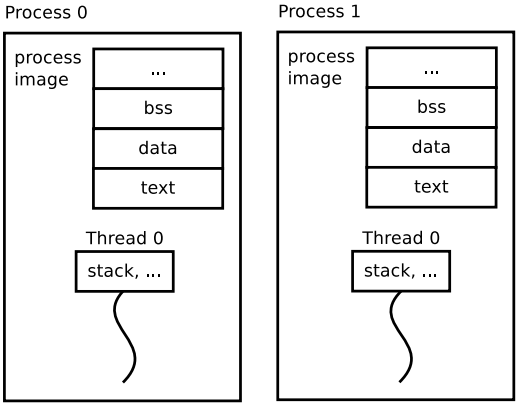
Reading and processing are parallelized!

(FYI) Processes vs. Threads 1/2

- Forking generates a new process with separate memory space
- Relatively expensive operation (lots of work for the OS)
- Cooperation requires IPC (pipes, shared memory, ...)

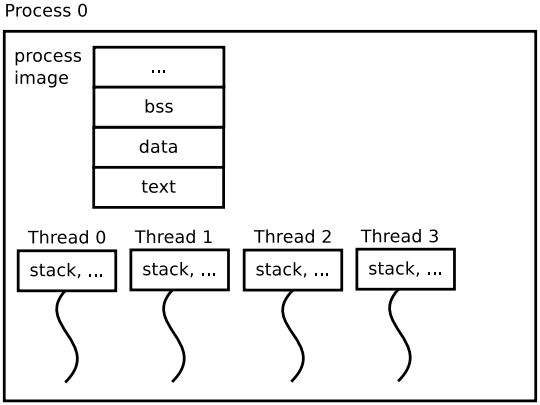
FYI 1: Parent can call `mmap(2)` with `MAP_SHARED|MAP_ANONYMOUS` to share unnamed memory regions with the child.

FYI 2: You can then create semaphores in the shared memory region for explicit synchronization.



(FYI) Processes vs. Threads 2/2

- Threads share the same memory space
- Relatively cheap for the OS
- Threads can work on the same variables, no IPC needed
- Still need synchronization (e.g., mutex)! But no need to create shared memory regions
- `man 7 pthreads`



- Michael Kerrisk: A Linux and UNIX System Programming Handbook, No Starch Press, 2010.
- man pages: `fork(2)`, `exec(3)`, `execve(2)`, `exit(3)`, `wait(3)`, `pipe(2)`, `dup(2)`